

通用 Ontology 平台 MVP 开发方案（V2.2）

版本：V2.2（2026-08-05） | 需求：逆向建模 → 多源整合 → 生成新系统 V2.2 变更：新增第 6 节「建模准确性方法论」（五关系类型 + 数据交集验证 + 评估体系），并同步更新 M3 设计。

1. 一句话目标

30 分钟演示链路：连两个老数据库 → AI 各自逆向生成本体草稿 → 人工确认并把两个系统的“客户”合并为一个对象 → 一键生成新系统骨架（新库表 + **CRUD API** + 可运行的管理界面），每个字段都能追溯回源表。

验证两个核心假设：

- 1. AI 辅助逆向建模能把“读懂老系统”的成本从月级压到天级
- 2. 本体作为“老系统拆迁重建的中间件”，客户愿意为“统一语义 + 生成新系统”付费

2. 需求确认结论（本方案的输入）

问题	结论
目标场景	企业已有存量系统，逆向建模与数据整合
第一优先用途	改造/重建系统：本体作为替换或整合老系统的中间蓝图
数据源	关系数据库（MySQL / PostgreSQL / Oracle）
正向生成产出物	可运行应用骨架：新库表 DDL + CRUD API + 管理界面
典型场景	整合为主：多老系统共存、先统一语义；整体替换是后续坡道

3. 竞品核查结论（2026-08 调研）

相邻赛道	现状	对我们的含义
AI 逆向建模	无成型产品：传统工具只出物理模型；SqlDBM 有 AI 语义	路径空白确认。LLM 建模准确率 68–86%，印证“草稿+人

相邻赛道	现状	对我们的含义
	命名但面向英文云数仓； DataWorks 靠表名规则且绑自家引擎；Ontop 语义强但零自动化	工确认”设计正确
MDM/实体解析	商业与国内 MDM 全部合并数据记录，无人把”模型层合并+映射留痕”做成产品	差异化真实存在；定位为”整合前期的语义对齐层”；护城河是裁决知识库与映射资产
代码生成器	现有生成器输入是技术模型/单库/自然语言；”语义模型→应用骨架”无人做，行业共识”AI 建模、模板出码”	壁垒在 M1-M4 的本体与血缘数据质量；窗口期有限

4. 明确不做（V1 红线）

不做项	后置到
Action 写回 / 行动 Agent	V2
数据迁移（golden record 记录层合并）与双跑切换	V2，V1 只生成迁移映射说明——与 MDM 的边界
查询 Agent（自然语言问数）	V1.5 附加能力
物化同步（CDC）	V2，V1 逆向建模只读源库
模板市场 / 多租户 / 细粒度权限	V3
图数据库	PostgreSQL 递归查询顶住

5. 功能清单（6 个模块）

#	模块	干什么	关键点
M1	连接器	连 MySQL/PG/Oracle，拉 schema + 采样数据	只读；Oracle 方言单独适配
M2	AI 逆向建模助手	LLM 读 schema → 生成本体草稿（YAML）	只产草稿不落库；建 3-5 个测试库的回归评测集
M3	多源整合工作台	跨源对象的五种关系类型裁决（合并/上位	差异化核心①，方法

#	模块	干什么	关键点
		对象/生命周期/子类型/不合并)，数据交集验证，决策留痕	论见第 6 节
M4	本体管理	YAML 存储、Git 版本化、React Flow 可视化	本体 = 单一事实源
M5	正向生成器	本体 → Alembic 迁移脚本 + FastAPI CRUD + Refine 管理界面，生成物携带字段级血缘标注	差异化核心②
M6	血缘追溯	新系统任一字段 → 本体属性 → 源表列的映射链查询与可视化	整合可信度与审计的基础

6. 建模准确性方法论（核心 know-how）

6.1 设计原点：准确性不是“像不像”，而是“对不对得起业务”

两个系统的模型交叉但不完全一样是常态（典型如招聘系统 vs HR 系统），“合并/不合并”的二元判断必然犯错。因此本产品的合并决策是关系类型裁决，准确性评估也分两层：单库逆向（M2）与跨源合并（M3）。

6.2 单库逆向建模的评估指标（M2）

指标	测量方式	说明
对象识别精确率/召回率	3-5 个测试库人工标注基准（该建的对象、该忽略的噪音表）	漏建与噪音分开统计
属性/关系映射准确率	与标注基准自动比对	显式外键易，隐式关系（无外键但逻辑关联）是难点
人工修改率（北星指标）	AI 草稿 → 人工确认稿的编辑距离	直接度量“AI 省了多少事”；每次真实使用都在产生数据，逐版本追踪
可执行性验证	用本体生成查询执行，对比手	唯一全自动的客观验证——模

指标	测量方式	说明
	写 SQL 结果	型对不对，跑出来的数据不会撒谎

6.3 跨源合并的五种关系类型（M3 的裁决选项）

关系类型	判定特征	建模方式	招聘 vs HR 案例
① 完全等价	同一业务概念、属性高度重合	合并为一个对象，多源 sources 映射	两系统的“部门”
② 部分重叠	公共属性 + 各自特有属性	建上位对象（如“人员”）承载公共属性，各自保留特有属性	候选人与员工共享姓名/身份证/电话
③ 生命周期阶段	同一实体的不同时期状态	统一对象 + 状态/角色属性，附阶段转化关系	候选人→员工（本案例最可能答案）
④ 子类型	一个是另一个的特化	接口/继承表达（V1 预留 Interface）	内部推荐候选人 C 候选人
⑤ 仅名字像	名称相近但语义不同	不合并，各自独立	招聘“职位(JD)” vs HR“岗位(编制)”

6.4 三层证据：把语义判断变成数据验证

层	证据	强度	操作
1. Schema 语义	表名/列名相似度、注释、采样模式	软证据	LLM 给出关系类型建议 + 理由，不定案
2. 数据交集	标识字段交集率（身份证/手机号/邮箱集合 $\cap$ ）	硬证据	只读采样即可算：交集率 $\approx 0\%$ 指向独立；30% 强烈指向③生命周期； $\approx 100\%$ 指向①/②
3. 人工裁决 + 业务测试	领域专家选择关系类型；合并后跑“业务测试问题集”验证	定案	测试问题如“查所有从候选人转正的员工”，答对才算通过；决策留痕可回滚

数据交集验证是关键手段：它把“语义像不像”变成“数据证明是什么关系”，且不搬数据、只读采样即可完成。

6.5 评估体系落地

- 1. M2 评测集：3-5 个行业测试库 + 人工标注基准本体，每次改 prompt 跑回归
- 2. M3 评测对：人工构造 5-8 对“交叉但不同”的系统（招聘/HR 为第一个案例），标注正确关系类型，追踪 LLM 建议准确率与人工裁决接受率
- 3. 北星指标：人工修改率 + 裁决接受率，逐版本对比

7. 最小元模型（V1 只实现 5 个概念）

```
# ontology.yaml 示例：两个老库整合后的“客户”
object_types:
  customer:
    label: 客户
    properties:
      - { name: id,      type: int,      pk: true }
      - { name: name,    type: string,   label: 姓名 }
      - { name: level,   type: enum,     values: [金, 银, 铜] }
    sources:                                     # 多源映射：一个对象对多个源表
      - { connection: old_erp, table: customers, pk: cust_id,
          fields: { name: cust_name, level: vip_flag } }
      - { connection: old_crm, table: t_client,  pk: client_no,
          fields: { name: client_name } }        # level 在 CRM 无对应，
标记缺失
link_types:
  - { name: placed, from: customer, to: order, via: { fk:
orders.customer_id } }
```

五种关系类型在元模型中的表达：①等价 = 单对象多 sources；②部分重叠 = 上位对象 + 各自对象（公共属性上移）；③生命周期 = 统一对象 + 状态属性 + 转化关系（link）；④子类型 = Interface（V1 预留，V2 实现）；⑤独立 = 无映射。Action、Function V1 不实现，YAML 预留扩展位。

8. 技术栈（现成轮子）

层	选型	理由
前端	Next.js + React Flow + shadcn/ui	本体可视化 + 整合裁决界面

层	选型	理由
后端	Python FastAPI + SQLAlchemy	多源方言适配与 LLM 编排生态
存储	PostgreSQL (本体定义 JSONB)	一个库顶所有
LLM	主流模型结构化输出	两用途：schema → 本体草稿；合并关系类型建议（给依据，裁决权在人）
出码	Jinja2 确定性模板，禁止 LLM 直接写代码	可重复可测试
DDL	Alembic 迁移脚本	可演进、可回滚
CRUD API	参考 fastapi-crudrouter 结构	分页/过滤/权限位预留
管理界面	Refine (或 React-Admin) 自动生成	精力留给血缘可视化差异点
交集验证	SQL 采样 + 集合交集计算	只读，标识字段自动识别（身份证/手机/邮箱正则）

核心表 8 张：connections、ontologies、ontology_versions、object_mappings（多源映射）、merge_decisions（裁决留痕：关系类型、证据快照、裁决人）、overlap_analyses（交集验证结果缓存）、api_keys、generation_logs。

9. 四周计划

周	交付	验收
W1	M1 连接器 (MySQL/PG 先行) + schema 读取	连上两个库，页面看到表结构
W2	M2 AI 逆向建模 + M4 YAML 存储/确认界面	10 表库生成本体草稿，人工改两处保存
W3	M3 整合工作台（五关系类型裁决 + 交集验证）+ M6 血缘	招聘/HR 案例：候选人 → 员工按③建模成功，交集率证据可查
W4	M5 正向生成器 (Alembic + CRUD + Refine) + 演示打磨	端到端 demo：生成的新应用跑通人员增删改查

10. 验收 demo 脚本

准备两个 mock 老库（“招聘系统”和“HR 系统”，人员相关表结构故意设计为交叉但不同）：

1. 配置两个连接（2 分钟）→ 2. AI 分别生成本体草稿（4 分钟）→ 3. M3 裁决：系统提示“候选人/员工交集率 34%，建议按生命周期阶段建模”，人工确认（8 分钟）→ 4. 一键生成新系统（即时）→ 5. 新系统人员管理页增删改查，点开“姓名”字段查看“来自招聘.candidate_name 和 HR.emp_name”，并跑通业务测试问题“查所有已转正人员”（全程 ≤30 分钟含讲解）

11. 三个最大风险（MVP 视角）

1. **LLM 逆向建模质量不稳**（实测 68–86%）：底线“只产草稿+人工确认”；评测集做回归
2. **关系类型判错**（如把⑤误判为①）：LLM 只给建议+证据快照，裁决权在人；merge_decisions 留痕可回滚，沉淀为映射资产
3. **被快速复制**：生成器与字段映射会商品化，护城河 = 本体质量 + 裁决知识库 + 血缘数据，以及第 6 节这套方法论的工程化深度